algorithm can make large steps and it can also move to worse positions, without making any use of any knowledge of the cost function (either previous or acquired while running), how do we know it will end up in the area of the global minimum, and not get stuck in some sort of funnel to a local minimum, when we start reducing temperature? Well, we don't. But, with most landscapes, if we run the algorithm long enough, and we find the right configuration, in practice it will get pretty close to the global optimum, and outmatch gradient descent.

There are lots of "ifs" in that proposition… the truth is that, as most heuristic algorithms, it works well in practice when it's in the hands of someone who knows how to tune it properly, and have enough computing time to make it run for lots of iterations.

Still, if that's the case, then simulated annealing is a great alternative for scenarios where gradient descent would suffer: it's not that one is better than the other, just (like all tool) there are problems for which gradient descent works better, and other where it would hard or impossible to apply it.

But let's see in details why it does work.

The key of the algorithm is the probabilistic mechanism that allows to move uphill (lines #2-#5), towards worse values for the cost function, and in particular the function that spits out the probability of accepting a positive delta, depending on temperature and on the magnitude of this delta.